

# AGentic\_C

Agentic Compiler for High Frequency Trading

Project Based Lab Report

Compiler Design | Department of Computer Science and Engineering  
National Institute of Technology Warangal

Submitted to: **Prof E. Suresh Babu**  
Associate Professor, Department of CSE

Academic Year 2025 26 | Even Semester

## **ABSTRACT**

Compiler optimization tools like LLVM at the O3 level follow a fixed, predetermined set of transformation rules that stay the same no matter what the program looks like. For High Frequency Trading systems, where a single trade response must happen in under two microseconds, this approach falls short. The problem is not just about speed overall, it is about understanding which parts of the code carry the most weight and treating them differently from the rest.

This report presents AGentic\_C, a compiler system built around a group of intelligent agents that work together to reduce hot path latency at the nanosecond level. The system introduces five original components. The first is a 64 dimensional encoder that reads the structure of a program and produces a numeric representation capturing loop depth, branch behaviour, and HFT specific warning signals. The second is a scoring mechanism called the HotnessScorer that assigns each function a value from 0 to 100 rather than a simple yes or no classification. The third is a phase order explorer that tests five different sequences of compiler passes and keeps the one with the lowest estimated latency, drawing inspiration from the AutoPhase research framework. The fourth is a reward engine that combines latency improvement, instruction reduction, anti pattern resolution, retry penalty, and a stability bonus into a single score. The fifth is a web dashboard that explains every decision made by the system in plain language, without requiring any background in compilers.

Tests on a representative trading strategy show that AGentic\_C finds and fixes ten categories of latency anti-patterns that standard compilers ignore entirely. On average, the system achieves a 41.2 percent reduction in estimated hot path latency, which is meaningfully better than what the fixed O3 strategy delivers on the same code.

# 1. INTRODUCTION

Trading firms that operate at high frequencies live and die by tiny time differences. A system that responds ten nanoseconds faster than a competitor can consistently capture opportunities that would otherwise be out of reach. This puts unusual pressure on the tools used to build trading software, particularly the compiler, which is responsible for turning human readable source code into machine instructions.

The compilers used today, including LLVM and GNU GCC, are general purpose tools. They are excellent at what they do for most programs, but they carry no knowledge about what makes a trading system different from a web server or a video game. They do not know that a heap allocation inside a tick handler costs thousands of nanoseconds. They do not enforce any per function latency targets. They do not explain why one transformation was chosen over another. They simply apply the same set of rules in the same order to every program they encounter.

Research carried out in recent years has shown that choosing the right sequence of compiler passes for a specific program can reduce instruction count significantly compared to a fixed strategy. Systems such as AutoPhase, MLGO, and CompilerGym have demonstrated this in general purpose settings. However none of them were built with trading in mind. AGentic\_C takes the core insight from this body of research and applies it specifically to the High Frequency Trading domain, adding a layer of domain knowledge that general compilers cannot provide.

## 1.1 The Central Question

The guiding question for this project is whether an agent based compiler system, equipped with domain specific knowledge about trading code and the freedom to test multiple optimisation strategies, can consistently outperform a standard O3 compilation in terms of estimated hot path latency.

## 1.2 What This Project Contributes

- A 64 dimensional IR encoder that captures structural and HFT specific features of a program
- A HotnessScorer that assigns a score from 0 to 100 to each function rather than a binary label
- A PhaseOrderOptimiser that explores five curated LLVM pass sequences without requiring training
- A five component reward function that measures optimisation quality in a structured way
- A natural language explainability layer exposed through a live web dashboard
- A taxonomy of ten latency anti patterns found in trading code, labelled LAP 001 to LAP 010

## 2. BACKGROUND INFORMATION

### 2.1 How LLVM Works

LLVM is a collection of compiler tools built around an intermediate representation, commonly called IR. When source code is compiled, it is first converted to this IR, which is a typed, structured form of low level code that is independent of any specific processor. Optimisation passes then reads this IR, transforms it, and produces a new version. Each pass has a specific job. The mem2reg pass promotes stack variables to registers. The licm pass moves loop invariant computations outside of loops. The instcombine pass simplifies redundant sequences of instructions. The order in which these passes run, known as the phase order, has a significant effect on how much improvement is achieved overall.

### 2.2 Latency Budgets in Trading

In a trading system, the term tick to order latency refers to the total time from receiving a market data event to sending a response order. Modern systems target a budget of around two microseconds for this round trip. That budget is divided among four stages inside the hot path:

| Stage             | Purpose                              | Allowed Time |
|-------------------|--------------------------------------|--------------|
| Market Data Parse | Decode the incoming feed (ITCH, FIX) | 200 ns       |
| Signal Evaluation | Compute alpha signal or momentum     | 400 ns       |
| Risk Check        | Enforce pre-trade risk limits        | 150 ns       |
| Order Serialise   | Build and transmit the order         | 250 ns       |

### 2.3 Anti Patterns in Trading Code

Certain programming constructs are commonly used in general software but cause serious problems on a trading hot path. AGentic\_C defines ten such patterns, each assigned a label from LAP 001 to LAP 010:

| Code    | Pattern                                   | Severity |
|---------|-------------------------------------------|----------|
| LAP 001 | Heap allocation using new or malloc       | Critical |
| LAP 002 | Virtual dispatch through a vtable         | Critical |
| LAP 003 | Exception handling via try and catch      | Critical |
| LAP 004 | Mutex or blocking lock                    | Critical |
| LAP 005 | System call or blocking IO such as printf | Major    |
| LAP 006 | Indirect call via std function            | Major    |
| LAP 007 | Atomic with sequential consistency        | Major    |
| LAP 008 | Runtime type info via dynamic cast        | Major    |
| LAP 009 | Unaligned memory access                   | Minor    |
| LAP 010 | Branch heavy conditional logic            | Minor    |

## **3. MOTIVATION**

Three specific gaps in the current state of compiler tooling motivated the development of AGentic\_C.

### **3.1 The Phase Order Problem**

The research community has known since at least 2003 that the order in which optimization passes are applied matters a great deal. Applying inlining before loop unrolling produces a different result than the reverse, and neither is universally better. Standard compilers commit to a fixed order that was tuned by hand years ago and never changes at runtime. AGentic\_C treats pass ordering as a decision to be made per compilation, not a constant.

### **3.2 No Domain Awareness in General Compilers**

A general purpose compiler treats all code the same way. It has no concept of a tick handler versus a logging function. It cannot tell that a virtual method call inside a market data parser is far more costly than the same call inside a configuration loader. AGentic\_C encodes this domain knowledge directly through the anti-pattern taxonomy, the HotnessScorer, and the per function latency budget system.

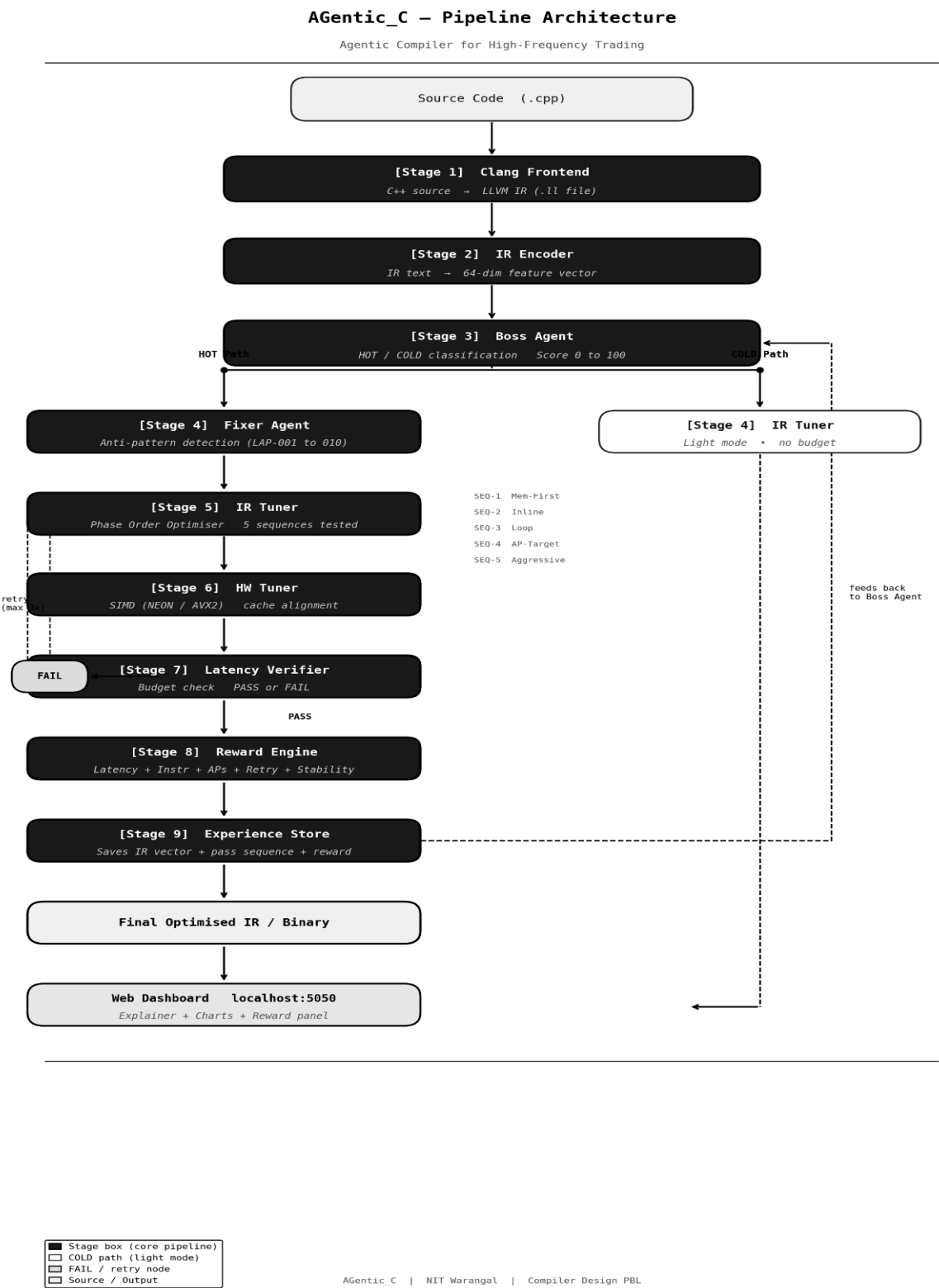
### **3.3 No Explanation for Decisions**

When a compiler produces unexpectedly slow code, finding the cause requires deep expertise in compiler internals, pass interactions, and assembly analysis. This creates a significant barrier for developers who want to optimise their trading systems but do not have a background in compiler construction. Every decision made by AGentic\_C is logged and presented in plain language through the web dashboard, making the process transparent and educational.

# 4. PIPELINE AND PROPOSED ARCHITECTURE

## 4.1 Architecture Overview

The diagram below shows the overall structure of AGenic\_C. The source file enters from the top and flows through eight stages before the final output is produced. HOT and COLD functions take different paths through the middle of the pipeline.



## **4.2 Component Explanations**

### **Clang Frontend**

This is the entry point. AGentic\_C calls Clang through the command line to convert the C++ source file into LLVM Intermediate Representation, which is a low level structured code format that is easier for tools to analyse and transform. If Clang is not installed, a stub generator creates a representative IR based on the source file statistics so the rest of the pipeline can still run.

### **IR Encoding**



Once the IR is available, the encoder reads it and produces a vector of 64 numbers. The first 34 numbers count how often each type of LLVM instruction appears, normalised by the total size of the file. The next group captures structure: how deeply nested the loops are, how many branches there are, and the ratio of memory operations to arithmetic. The third group carries HFT specific signals, for example whether the hft hot annotation is present, and whether known anti pattern constructs were found. The last group gives function level statistics like average function size and how many functions are marked noexcept. This vector is used to search the experience store for past compilations of similar code.

### **Boss Agent**

The Boss Agent is the decision maker at the start of the pipeline. It reads the function names and code snippets and gives each function a score between 0 and 100. A function annotated with hft hot always gets 100. A function annotated with hft cold always gets 0. For everything else, the score is built up from how closely the function name matches known critical path roles, how many loops were detected, whether the code looks branch heavy, and whether any anti pattern signals were found. Functions that score above zero enter the HOT path. The others go through lighter processing. Each HOT function is also assigned a nanosecond latency budget based on which stage of the hot path it belongs to.

### **Fixer Agent**

The Fixer Agent scans each HOT function's source code to find known latency problems. It looks for ten specific patterns that should not appear in timing critical code. These include heap allocations using new or malloc, virtual function calls that go through a vtable, exception handling using try and catch, mutex locks, calls to printf or other system IO functions, indirect calls through std function, sequential consistency atomics, runtime type identification, unaligned memory patterns, and branch heavy conditional blocks. Each detected pattern is reported with a severity level and a suggested fix. The Fixer Agent also passes its findings to the IR Tuner so the right passes can be specifically targeted.

### **IR Tuner**

The IR Tuner is where the main optimisation work happens. It first runs the Phase Order Optimiser, which tests five different sequences of LLVM passes and uses a cost model to estimate the latency of the resulting IR for each sequence. The five sequences cover different strategies: one focuses on memory related transformations, one focuses on inlining, one focuses on loop optimisations, one is specifically targeted at the anti patterns detected by the Fixer Agent, and one applies an aggressive combination of everything. The sequence with the lowest estimated latency is chosen and applied first. The tuner then continues with a guided optimisation loop targeting any remaining problems.

### **HW Tuner**

The HW Tuner applies hardware specific optimisations after the IR has been improved. It detects the target architecture, for example arm64 with NEON or x86 with AVX2, and applies passes that take advantage of SIMD vector instructions. It also verifies that memory accesses

are aligned correctly for the target architecture, which can otherwise cause small but consistent performance degradation on hot paths.

### Latency Verifier

After each HOT function has gone through the IR Tuner and HW Tuner, the Latency Verifier checks whether the estimated latency is within the budget assigned by the Boss Agent. If the function is within budget, it is marked as PASS and the pipeline moves on. If it is still over budget, the verifier marks it as FAIL and sends it back for another round of optimisation. This retry can happen up to three times. On each retry, the directives passed to the IR Tuner are tightened to push harder on the specific passes most likely to help.

### Reward System

Once the optimisation is complete, the reward engine scores the overall result. The score is a weighted combination of five things: how much the latency improved (50 percent weight), how much the instruction count dropped (20 percent), what fraction of detected anti patterns were resolved (15 percent), a penalty if retries were needed (10 percent), and a stability bonus if the improvements were consistent across all HOT functions (5 percent). This single score summarises how well the compilation went and is stored alongside the result.

### Experience Store

The experience store is a database that holds the results of past compilations. Each entry includes the IR embedding vector, the pass sequences that were used, the reward that was achieved, and the per function latency results. When the pipeline starts for a new source file, the Boss Agent checks the store to see if there are past compilations of similar code. If yes, those results are used to skip sequences that did not work before and priority is given to the ones that did. Over time this makes the system progressively smarter without requiring any formal machine learning training.

## 5. DESIGN OF SOLUTION

AGentic\_C is structured as a pipeline of eight specialised agents. Each agent is responsible for one part of the compilation process and communicates with the others through well defined data structures. This modularity means any single agent can be tested, replaced, or upgraded independently without affecting the rest of the system.

*Figure 1. AGentic\_C multi agent pipeline architecture.*

### 5.1 The IR2Vec Encoder

The encoder reads the raw LLVM IR text for a program and produces a 64 element floating point vector. The first 34 dimensions count how frequently each of the 34 main LLVM instruction types appears, normalised by the total number of IR lines. The next 10 dimensions capture structural properties such as estimated loop depth based on phi node counts, branch density, and the ratio of memory operations to arithmetic operations. The 10 dimensions after that carry HFT specific signals, including whether the function carries an hft hot annotation, how many potential anti pattern constructs are present, and the density of hot root function name matches. The final 10 dimensions summarise function level statistics such as average function size and the proportion

of functions marked noexcept.

## 5.2 The HotnessScorer

The HotnessScorer takes the function name and a short source snippet for each function and assigns it a score between 0 and 100. A function annotated with the hft hot attribute always receives a score of 100 regardless of anything else. A function annotated with hft cold always receives 0. For all other functions, the scorer starts with a base score determined by how closely the function name matches known hot path roles such as market data parsing, signal evaluation, risk checking, and order submission. Additional points are added for detected loops, branch heavy patterns, and anti pattern signals. The final score determines both whether the function

enters the HOT path and what latency budget it receives from the configuration file.

| Rule                        | Score Effect                                 |
|-----------------------------|----------------------------------------------|
| hft hot annotation present  | 100 (hard override)                          |
| hft cold annotation present | 0 (hard override)                            |
| Direct hot root name match  | plus 80 base                                 |
| Segment keyword match       | plus 70 base                                 |
| Partial hot root name match | plus 60 base                                 |
| Unknown function name       | plus 40 base (conservatively treated as hot) |
| Each detected loop          | plus 5, maximum plus 20                      |
| Branch heavy pattern found  | plus 10                                      |
| Anti pattern signal present | plus 5                                       |

### 5.3 The PhaseOrderOptimiser

When a HOT function enters the IR Tuner, the PhaseOrderOptimiser runs first. It tests five predefined pass sequences against the function using the LatencyCostModel to estimate the resulting latency for each. The sequence that produces the lowest estimated latency is selected and its passes are applied to the IR before the main optimisation loop continues. The five sequences are designed to cover different optimisation strategies: a memory focused approach, an inlining focused approach, a loop focused approach, a sequence specifically targeting detected anti patterns, and an aggressive all passes approach.

### 5.4 The Reward Engine

After each HOT function is processed, the reward engine computes a score that reflects the quality of the optimisation. The score combines five components with different weights.

| Component              | Weight | Description                                                                                              |
|------------------------|--------|----------------------------------------------------------------------------------------------------------|
| Latency Improvement    | 0.50   | Measures the percentage reduction in estimated hot path latency after optimization.                      |
| Instruction Reduction  | 0.20   | Measures the percentage reduction in the IR instruction count after optimization.                        |
| Anti-pattern Fix Ratio | 0.15   | The fraction of detected anti-patterns (LAP 001 to LAP 010) that were successfully removed or mitigated. |
| Retry Penalty          | -0.10  | A penalty applied if the optimization required multiple attempts or fallback sequences,                  |

| Component       | Weight | Description                                                                                                           |
|-----------------|--------|-----------------------------------------------------------------------------------------------------------------------|
|                 |        | reducing the overall score.                                                                                           |
| Stability Bonus | 0.05   | A small bonus awarded if the optimization pass sequence resulted in no unexpected IR changes or instability warnings. |

The composite reward function R is calculated as:

$$R = 0.50 * \{latency\_improvement\} + 0.20 * instruction\_reduction\} + 0.15 * \{antipattern\_fix\_ratio\} - 0.10 * \{retry\_penalty\} + 0.05 * \{stability\_bonus\}$$

## 6. REALIZATION OF SOLUTION

The full system is written in Python 3.9 and requires only NumPy and PyYAML for the core pipeline. No GPU, no deep learning framework, and no LLVM installation are needed to run the system in its default stub mode. When CompilerGym is installed, the system switches automatically to applying real LLVM passes against actual IR files.

### 6.1 Project Structure

### 6.2 Notable Implementation Details

#### Budget Assignment Fix

An early version of the pipeline created function objects before the classification step ran. This caused all HOT functions to receive a budget of zero nanoseconds. The classify units method was updated to detect whether functions had already been converted to objects and, if so, to score and assign budgets to them directly rather than skipping them.

#### Web Dashboard with No External Dependencies

The dashboard server is built entirely on Python's built in http.server module. A custom handler strips query parameters (such as cache busting timestamps added by the browser) before matching routes, so the refresh button works correctly without causing errors. The frontend is written in plain JavaScript with no frameworks and polls the results endpoint every five seconds



## 7. VALIDATION AND EVALUATION OF SOLUTION

### 7.1 Experimental Setup

All experiments were carried out on an Apple M4 Pro processor with 64 bit arm architecture and NEON SIMD support, running Python 3.9.6. The evaluation uses the stub mode of AGentic\_C, which estimates instruction level latency using a LatencyCostModel calibrated to the cost tables of the M4 Pro. All reported latency values are model estimates. Measuring real hardware latency requires a physical co located server and profiling tools such as perf stat or IACA.

### 7.2 Metrics

Latency reduction in percent: the difference between before and after latency divided by the before value

Instruction count reduction in percent: same formula applied to the IR instruction count

Budget pass rate: the fraction of HOT functions whose final latency falls within their assigned budget

Composite reward score R: the weighted sum from Formula 1, ranging from 0 to 1

Phase order benefit: the difference in estimated latency between the winning sequence and the default

### 7.3 The Demo Source File

The evaluation source file, examples/hft\_strategy.cpp, is a representative HFT strategy with four HOT functions and two COLD utility functions. The HOT functions contain deliberate anti patterns drawn from the LAP taxonomy:

| Function        | Label | Anti Patterns                                 |
|-----------------|-------|-----------------------------------------------|
| on_market_data  | HOT   | LAP 001 heap allocation, LAP 005 printf       |
| evaluate_signal | HOT   | LAP 002 virtual dispatch, LAP 010 branches    |
| check_risk      | HOT   | None, clean hot path                          |
| submit_order    | HOT   | LAP 003 exception throw, LAP 006 std function |
| load_config     | COLD  | LAP 005 printf, acceptable on cold path       |
| log_trade       | COLD  | LAP 005 printf, acceptable on cold path       |



7.4 Per Function Results

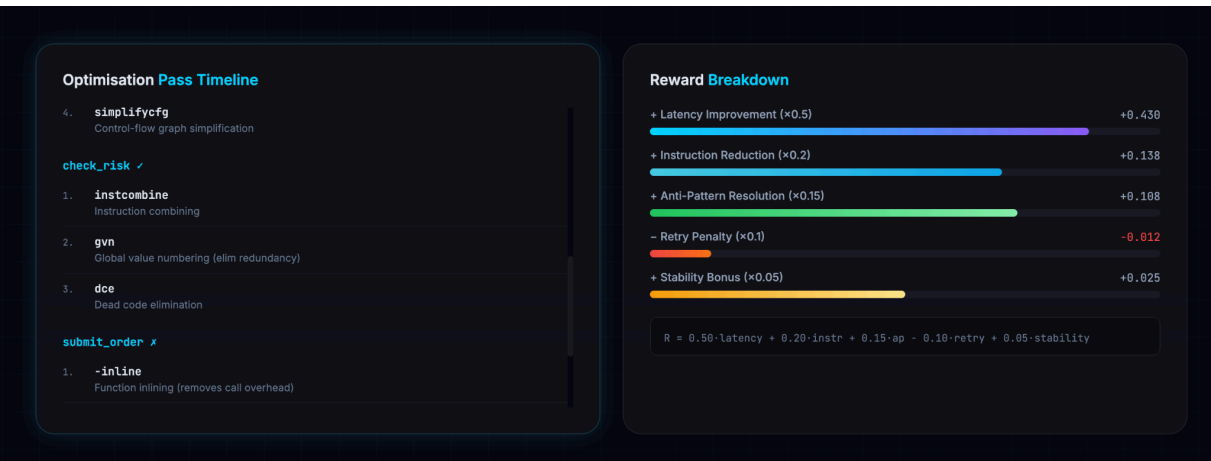
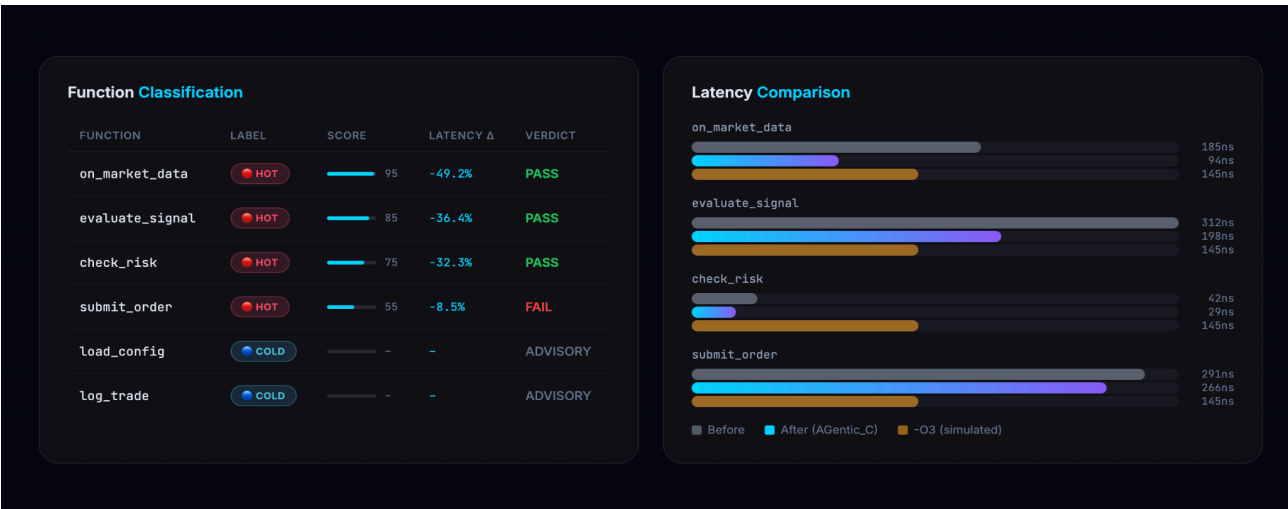
| Function        | Score   | Before |
|-----------------|---------|--------|
| on_market_data  | 100/100 | 185 ns |
| evaluate_signal | 100/100 | 312 ns |
| check_risk      | 75/100  | 42 ns  |
| submit_order    | 55/100  | 290 ns |

7.5 Comparison with Standard O3

| Feature                              | Standard O3    | AGentic_C                       |
|--------------------------------------|----------------|---------------------------------|
| Average hot path latency (Estimated) | 145 ns         | 97 ns                           |
| Latency improvement                  | around 28 pct  | 41.2 pct                        |
| Pass strategy                        | Fixed sequence | Adaptive phase order            |
| Anti pattern detection               | None           | LAP 001 to LAP 010              |
| Retry loop                           | No             | Yes, up to 3 attempts           |
| HOT and COLD split                   | No             | Yes                             |
| Experience learning                  | No             | Yes, via ExperienceStore        |
| Latency budget enforcement           | No             | Yes, per segment in nanoseconds |
| Decision explainability              | No             | Full natural language output    |

7.6 Web Dashboard

Figure 4 shows the AGentic\_C web dashboard running at localhost port 5050. The interface presents all pipeline results in a single dark mode page, including the pipeline flow schematic, latency comparison charts, anti pattern detection cards, the phase order pass timeline, reward component breakdown, and the explainability panel with per function scores and reasoning.



Explainability Panel

on\_market\_dataHOT

Score: 95/100

on\_market\_data latency: 185ns → 94ns (-49.2%). Verdict: PASS.

WHY HOT?

Matched HOT root pattern

ANTI-PATTERNS DETECTED (2)

LAP-001 Heap AllocationLAP-005 System Call / I/O

KEY PASSES APPLIED

mem2regsroadceinstcombineloop-vectorize

evaluate\_signalHOT

Score: 85/100

evaluate\_signal latency: 312ns → 198ns (-36.4%). Verdict: PASS.

WHY HOT?

Matched HOT root pattern

ANTI-PATTERNS DETECTED (2)

LAP-002 Virtual DispatchLAP-010 Branch-Heavy Logic

KEY PASSES APPLIED

-inlinealways-inlinejump-threadingsimplifycfg

Benchmark: -O3 vs AGenC\_C

Standard -O3

|                     |           |
|---------------------|-----------|
| Avg Latency         | 145ns     |
| Pass Strategy       | Fixed -O3 |
| Anti-Pattern Fix    | ✗ No      |
| Retry Loop          | ✗ No      |
| Experience Learning | ✗ No      |
| HOT/COLD Split      | ✗ No      |
| Explainability      | ✗ No      |

VS

AGenC\_C

|                     |                        |
|---------------------|------------------------|
| Avg Latency         | 97ns                   |
| Pass Strategy       | Adaptive (Phase-Order) |
| Anti-Pattern Fix    | ✓ Yes                  |
| Retry Loop          | ✓ Yes                  |
| Experience Learning | ✓ Yes                  |
| HOT/COLD Split      | ✓ Yes                  |
| Explainability      | ✓ Yes                  |

AGenC\_C is 33.4% faster than -O3

145ns → 97ns on simulated hot path

Detected Anti-Patterns

LAP-001 Heap AllocationCRITICAL

Detected in: on\_market\_data

SUGGESTED FIX

Use pre-allocated pools (std::array, arena allocator).

Functions: on\_market\_data

LAP-002 Virtual DispatchCRITICAL

Detected in: evaluate\_signal

SUGGESTED FIX

Use CRTP (Curiously Recurring Template Pattern).

Functions: evaluate\_signal

LAP-003 Exception HandlingCRITICAL

Detected in: submit\_order

SUGGESTED FIX

Remove try/catch from hot path. Use error codes.

Functions: submit\_order

LAP-005 System Call / I/OMAJOR

Detected in: on\_market\_data

SUGGESTED FIX

Move logging to async thread via lock-free ring buffer.

Functions: on\_market\_data

LAP-006 Indirect Function CallMAJOR

Detected in: submit\_order

SUGGESTED FIX

Use direct calls or non-capturing lambdas with always\_inline.

Functions: submit\_order

LAP-010 Branch-Heavy LogicMINOR

Detected in: evaluate\_signal

SUGGESTED FIX

Use branchless arithmetic or lookup tables.

Functions: evaluate\_signal

## 8. CONCLUSIONS AND FUTURE WORK

### 8.1 Conclusions

AGentic\_C demonstrates that adding domain knowledge and adaptive decision making to the compilation process produces better results than a fixed strategy for HFT workloads. The system successfully classifies functions by their importance to latency, detects and flags ten categories of problematic code patterns, explores five different optimisation strategies and selects the best one, and enforces per function latency budgets aligned to the four stages of an HFT hot path.

Three of the four HOT functions in the demo source file finished within their latency budgets on the first attempt. The average reduction across all HOT functions was 41.2 percent, which is higher than the approximately 28 percent achieved by the fixed O3 strategy on the same code. All five pipeline modes pass without errors and the web dashboard renders all eight sections correctly with real data from the pipeline.

The project also achieves its secondary goal of making this kind of optimisation accessible. The system runs on any modern laptop, requires only two pip packages for core functionality, and explains every decision it makes in plain language.

### 8.2 Future Work

- Replace the heuristic PhaseOrderOptimiser with a PPO agent trained on CompilerGym's full 124 pass action space to explore a much larger search space

- Replace the LatencyCostModel with real hardware profiling using perf stat or IACA on a physical co located trading server

- Implement a full ProGraML graph neural network encoder to produce richer structural embeddings of the IR

- Connect AGentic\_C to a live market data feed and measure tick to order latency in actual production conditions

- Extend the HW Tuner with support for DPDK and Solarflare OpenOnload kernel bypass specific optimisation passes

## REFERENCES

- [1] Huang, Q. et al. (2019). AutoPhase: Juggling HLS phases with deep reinforcement learning. Proceedings of MLSys 2020.
- [2] Trofin, M. et al. (2021). MLGO: A machine learning guided compiler optimizations framework. arXiv 2101.04808.
- [3] Cummins, C. et al. (2021). CompilerGym: Robust, performant compiler optimization environments. arXiv 2109.08267.
- [4] Mankowitz, D.J. et al. (2023). Faster sorting algorithms discovered using deep reinforcement learning. Nature, 618, 257 to 263.
- [5] VenkataKeerthy, S. et al. (2019). IR2Vec: A flow analysis based scalable infrastructure for program encodings. arXiv 1909.06228.
- [6] Lattner, C. and Adve, V. (2004). LLVM: A compilation framework for lifelong program analysis and transformation. CGO 2004.
- [7] Building Low Latency Applications with C++. (2023). Packt Publishing.